

AWS Architecture and Database Service

Prof. Arnab Mandal

Assistant Professor

CSE Department

Content

1. AWS Well-Architected Framework Design Principles.....	3
2. Six Pillars of the AWS Well-Architected Framework.....	4
3. Amazon RDS	12
4 MySQL on Amazon RDS.....	14
5. PostgreSQL on Amazon RDS	15
6 Amazon RDS	17
8. Amazon Aurora	18
9. Amazon Dynamo	19
12.Amazon Redshift	20
13. Lunch an Amazon RDS DB instance	21

AWS Well-Architected Framework Design Principles :

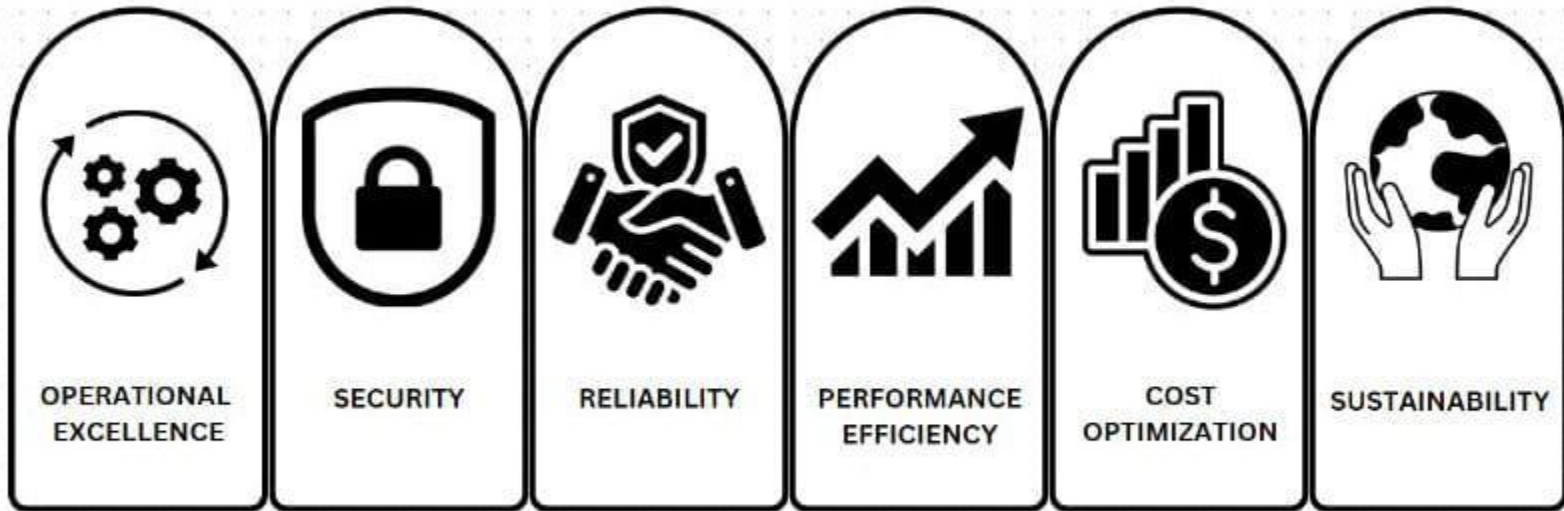
The AWS Well-Architected Framework offers best practices to design secure, high-performing, resilient, and cost-effective cloud architectures on AWS. It helps teams evaluate workloads consistently and improve their designs as applications grow and scale over time.

Benefits:

1. Assists architects and developers in building reliable and efficient AWS workloads.
2. Enables consistent evaluation of architectures against AWS best practices.
3. Identifies risks and areas needing improvement in cloud designs.
4. Provides guidance for building, scaling, and maintaining AWS environments effectively.

Six Pillars of the AWS Well-Architected Framework:

1. **Operational Excellence**
2. **Security**
3. **Reliability**
4. **Performance Efficiency**
5. **Cost Optimization**
6. **Sustainability**



Operational Excellence

The Operational Excellence pillar emphasizes running workloads effectively and efficiently while gaining insights into operational processes. It helps organizations continuously improve procedures to deliver better business value by optimizing operations, increasing efficiency, and enhancing overall performance.

Design Principles:

- **Perform Operations as Code:** Automate procedures to reduce errors and ensure consistency.
- **Make Frequent, Small Reversible Changes:** Apply small, reversible updates to minimize risk.
- **Refine Operations Procedures Frequently:** Continuously improve processes.
- **Anticipate Failure:** Plan for failures to keep systems resilient.
- **Learn from All Operational Failures:** Analyze failures to prevent them in the future.

Security:

The Security pillar focuses on protecting data, systems, and assets by applying best practices across all security areas. It emphasizes risk assessment and mitigation to securely operate workloads.

Design Principles:

- 1. Implement a Strong Identity Foundation:** Use AWS IAM to enforce strict access controls.
- 2. Allow Traceability:** Log, monitor, alert, and audit activities for full traceability.
- 3. Apply Security at All Layers:** Secure network, compute, and storage layers.
- 4. Automate Security Best Practices:** Use automated tools to scale security efficiently.
- 5. Protect Data in Transit and at Rest:** Encrypt data both during transfer and storage.
- 6. Keep People Away from Data:** Limit human access by automating processes.
- 7. Prepare for Security Events:** Conduct incident response drills and use automation to speed detection and investigation.

Reliability

The Reliability pillar ensures workloads continue to operate as expected despite failures by using automated recovery and disruption mitigation.

Design Principles:

- 1. Automatically Recover from Failure:** Use automation to detect and fix failures without manual intervention.
- 2. Test Recovery Procedures:** Regularly test recovery to ensure systems work as intended.
- 3. Scale Horizontally:** Distribute workloads across multiple instances to boost availability.
- 4. Stop Guessing Capacity:** Automatically scale resources based on demand.
- 5. Manage Change with Automation:** Use automation to deploy and manage system changes.

Performance Efficiency

Focuses on using IT and computing resources efficiently to meet system needs and maintain that efficiency as demand and technology evolve.

Design Principles:

1. **Democratize Advanced Technologies:** Use technologies as services to simplify implementation.
2. **Go Global in Minutes:** Deploy across multiple AWS Regions for lower latency and better user experience.
3. **Use Serverless Architectures:** Automatically scale and optimize resources based on demand.
4. **Experiment More Often:** Test different architectures to improve performance.
5. **Consider Mechanical Sympathy:** Choose services that best fit your architecture and requirements.

Cost Optimization

Focuses on understanding, controlling, and reducing costs by scaling resources to meet business needs without overspending.

Design Principles:

- 1. Implement Cloud Financial Management:** Invest in managing cloud costs effectively.
- 2. Adopt a Consumption Model:** Pay only for what you use.
- 3. Measure Overall Efficiency:** Continuously monitor and optimize resource use.
- 4. Avoid Undifferentiated Heavy Lifting:** Eliminate unnecessary work that doesn't add value.
- 5. Analyze and Attribute Expenditure:** Track costs and assign them to specific workloads or teams.

Sustainability

The sustainability pillar tracks your progress toward policies and best practices that support a more sustainable future and not just a simple checklist.

Design Principles:

- 1. Understand your impact:** Understand your cloud infrastructure's environmental, social and economic impact.
- 2. Establish sustainability goal:** Plan your growth and design your architecture to reduce the impact per unit of work as per user.
- 3. Maximize utilization:** Right sizing each workload
4. Anticipate and adopt new, more efficient hardware and software offerings
5. Use managed services
6. Reduce the aftereffects of your cloud workloads

2: Amazon RDS (Relational Database Service)

Amazon Relational Database Service (Amazon RDS) is a managed database service provided by AWS that simplifies the setup, operation, and scaling of relational databases in the cloud. It supports popular engines such as MySQL, PostgreSQL, MariaDB, Oracle, and SQL Server, along with AWS's own Aurora database engine.

With RDS, AWS handles administrative tasks like:

- Database provisioning
- Backups and patching
- Monitoring and scaling
- Security (encryption and IAM integration)

This allows developers and organizations to focus on application logic instead of infrastructure management.

Key Features of Amazon RDS

1. **Managed Service:** Automates database administration tasks like backups, patching, and monitoring.
2. **Scalability:** Easily scale compute and storage resources up or down as needed.
3. **High Availability:** Supports Multi-AZ deployments for automatic failover and enhanced reliability.
4. **Automated Backups:** Provides automated backups with point-in-time recovery.
5. **Security:** Supports encryption at rest and in transit, IAM integration, and network isolation via VPC.
6. **Multi-Engine Support:** Compatible with several database engines, including MySQL, PostgreSQL, MariaDB, Oracle, and SQL Server.
7. **Performance:** Offers provisioned IOPS for consistent, low-latency performance.
8. **Monitoring and Metrics:** Integrated with Amazon CloudWatch for real-time monitoring and alerts.
9. **Easy Migration:** Supports database migration tools for easy data transfer to AWS.
10. **Cost-Effective:** Pay-as-you-go pricing with options for reserved instances to save costs.

MySQL on Amazon RDS

Amazon RDS for MySQL is a managed relational database service that makes it easy to set up, operate, and scale MySQL databases in the cloud. It automates common database administration tasks, allowing you to focus on application development.

Key Features:

- 1. Managed Service:** Automated backups, patching, monitoring, and scaling.
- 2. High Availability:** Multi-AZ deployments for automatic failover and improved durability.
- 3. Performance:** Supports provisioned IOPS for high and consistent throughput.
- 4. Security:** Encryption at rest and in transit, IAM authentication, and network isolation via Amazon VPC.
- 5. Scalability:** Easily scale storage and compute resources with minimal downtime.
- 6. Compatibility:** Fully compatible with existing MySQL applications and tools.
- 7. Automated Backup and Recovery:** Supports point-in-time recovery and automated snapshots.
- 8. Integration:** Works seamlessly with other AWS services like Lambda, CloudWatch, and IAM.

PostgreSQL on Amazon RDS

Amazon RDS for PostgreSQL is a managed relational database service that simplifies the setup, operation, and scaling of PostgreSQL databases in the cloud. It automates routine database tasks, enabling developers to focus on building applications.

Key Features:

1. **Managed Service:** Automated backups, patching, monitoring, and scaling.
2. **High Availability:** Supports Multi-AZ deployments for automatic failover and increased reliability.
3. **Performance:** Supports provisioned IOPS for consistent and low-latency performance.
4. **Security:** Encryption at rest and in transit, IAM integration, and network isolation using Amazon VPC.
5. **Scalability:** Easily scale compute and storage resources with minimal downtime.
6. **Compatibility:** Fully compatible with PostgreSQL extensions and tools.
7. **Automated Backup and Recovery:** Includes point-in-time recovery and automated snapshots.
8. **Integration:** Works well with other AWS services like Lambda, CloudWatch, and IAM.

SQL Server on Amazon RDS

Amazon RDS for SQL Server is a managed relational database service that simplifies deploying, operating, and scaling Microsoft SQL Server databases in the cloud. It handles routine database management tasks so you can focus on application development.

Key Features:

- 1. Managed Service:** Automated backups, patching, monitoring, and maintenance.
- 2. High Availability:** Supports Multi-AZ deployments for automatic failover and improved uptime.
- 3. Performance:** Offers provisioned IOPS for high and consistent performance.
- 4. Security:** Supports encryption at rest and in transit, IAM integration, and network isolation via Amazon VPC.
- 5. Scalability:** Easily scale compute and storage resources with minimal downtime.
- 6. Compatibility:** Supports multiple SQL Server editions and versions, compatible with existing SQL Server tools and applications.
- 7. Automated Backup and Recovery:** Provides point-in-time recovery and automated snapshots.
- 8. Integration:** Works seamlessly with AWS services like Lambda, CloudWatch, and IAM.

Amazon RDS (Relational Database Service)

Amazon RDS is a fully managed cloud service that simplifies the setup, operation, and scaling of relational databases. It automates routine tasks such as backups, patching, and monitoring, allowing you to focus on application development.

Key Benefits:

- 1. Managed Service:** Handles database administration tasks automatically.
- 2. Scalability:** Easily scale compute and storage resources on demand.
- 3. High Availability:** Supports Multi-AZ deployments for automatic failover and enhanced reliability.
- 4. Security:** Provides encryption, network isolation, and access control.
- 5. Multi-Engine Support:** Compatible with popular database engines like MySQL, PostgreSQL, MariaDB, Oracle, and SQL Server.
- 6. Automated Backup and Recovery:** Enables point-in-time recovery and automated backups.
- 7. Performance:** Offers provisioned IOPS for consistent, low-latency performance.
- 8. Integration:** Works seamlessly with other AWS services for monitoring, security, and automation.

Amazon Aurora:

While RDS is great, Aurora is the cloud-native version of MySQL and PostgreSQL. It was rebuilt by AWS specifically to take advantage of the cloud's power.

Performance: Up to 5x faster than standard MySQL and 3x faster than standard PostgreSQL.

Storage Architecture: Aurora uses a distributed, self-healing storage system that automatically scales up to 128 TiB. It replicates your data 6 times across 3 Availability Zones by default.

Aurora Serverless v2: A major teaching point for 2026. It scales compute capacity (ACUs) up and down in fractions of a second based on application demand, making it perfect for unpredictable workloads.

Global Database: Allows a single Aurora database to span multiple AWS regions, providing disaster recovery and low-latency local reads for global users.

Amazon DynamoDB:

DynamoDB is a fully managed key-value and document database that delivers single-digit millisecond performance at any scale.

Key Features:

- **NoSQL and Schema-less:** You don't need to define a fixed schema before adding data.
- **Massive Scale:** Handles over 10 million requests per second; powers Amazon.com's shopping cart.
- **Partition Key:** Distributes data across storage for fast access.
- **Provisioned vs. On-Demand:**
 - *Provisioned* mode is cost-effective when you can predict traffic.
 - *On-Demand* mode automatically handles traffic spikes for flexibility.
- **DAX (DynamoDB Accelerator):** An in-memory cache that speeds up response times from milliseconds to microseconds.

3. Amazon Redshift:

Aurora is for "transactions" (buying a book), Redshift is for "analytics" (calculating the total sales of all books in the last year).

- **OLAP (Online Analytical Processing):** Optimized for complex queries on massive datasets (petabytes).
- **Columnar Storage:** Unlike RDS, which stores data in rows, Redshift stores data in columns. This makes searching for a specific metric (like "Sum of Prices") incredibly fast.
- **Redshift Serverless:** In 2026, students can run analytics without managing clusters. It automatically provisions the right amount of compute capacity.
- **Zero-ETL Integration:** Aurora can now send data to Redshift in near real-time without you having to write complex code to move the data.

Launch an Amazon RDS DB instance:

Phase 1: Creating the Database

1. **Open the RDS Console:** Log in to the AWS Management Console and search for **RDS**.
2. **Create Database:** Click the orange **Create database** button.
3. **Choose a database creation method:** Select **Standard create**. This allows students to see all the configuration options, which is better for learning.
4. **Engine options:** Select **MySQL**.
5. **Templates:** Select **Free Tier**. This automatically disables expensive features like Multi-AZ and high-performance storage that would otherwise cost money.
6. **Settings:**
 - **DB instance identifier:** Name it cloud-class-db.
 - **Master username:** Keep it as admin.
 - **Master password:** Have students create a password they will remember
7. **Instance configuration:** Under the Free Tier, it will default to a db.t3.micro or db.t4g.micro. Explain to students that this is a "burstable" instance type.

8. Instance configuration: Under the Free Tier, it will default to a db.t3.micro or db.t4g.micro. Explain to students that this is a "burstable" instance type.

9. Connectivity:

- **Public access:** Select Yes. (In a real production environment, we choose "No," but for a lab, this allows students to connect from the classroom computers without complex networking).
- **VPC security group:** Choose Create new and name it rds-student-sg.

10. Create Database: Scroll to the bottom and click Create database.

Lab: Creating an Amazon EBS Volume

Use this walkthrough to show students how to add a "virtual hard drive" to a Linux or Windows server.

Step 1: Create the Volume

1. Log in to the **AWS Management Console**.
2. Navigate to **EC2 -> Elastic Block Store -> Volumes**.
3. Click **Create volume**.
4. **Volume Settings:**
 - **Volume type:** Select **General Purpose SSD (gp3)** (best price/performance).
 - **Size:** Enter a small size for the lab, e.g., 2 GiB.
 - **Availability Zone: CRITICAL!** It must be the same AZ as your EC2 instance (e.g., ap-south-1a).
5. Click **Create volume**.

Phase 2: Configuring Security

Even with "Public Access" enabled, the AWS Firewall (Security Group) will block connections by default.

1. Wait for the status to change from **Creating** to **Available** .
2. Click on the DB identifier (cloud-class-db).
3. Under the **Connectivity & security** tab, click the link under **VPC security groups**.
4. Select the **Inbound rules** tab and click **Edit inbound rules**.
5. Add a rule:
 - **Type:** MySQL/Aurora (Port 3306).
 - **Source:** Select **My IP** .
6. Click **Save rules**.

Phase 3: Connecting to the Database

To prove the database is working, students need to connect to it using the **Endpoint**.

1. Copy the **Endpoint** from the RDS console (it looks like: cloud-class-db.xyz.ap-south-1.rds.amazonaws.com).
2. Open a database tool (like **MySQL Workbench**, **DBeaver**, or even the **AWS CloudShell**).
3. Enter the **Endpoint** as the Hostname, the **Port** (3306), and the **Username/Password** created in Phase 1.
4. Once connected, have them run their first SQL command:

```
CREATE DATABASE StudentTest;  
SHOW DATABASES;
```

KEY TAKEAWAYS :

- **Well-Architected Framework:** Design resilient systems by following the 6 Pillars and core principles like testing at production scale and automating to eliminate guesswork in capacity planning.
- **Amazon RDS:** Simplifies database management by handling patching and backups for popular engines like MySQL, PostgreSQL, and SQL Server, allowing you to focus on data rather than infrastructure.
- **Amazon Aurora:** Provides an enterprise-grade, cloud-native relational database that offers significantly higher performance and automatic storage scaling compared to standard MySQL or PostgreSQL.
- **Amazon DynamoDB:** Delivers consistent, single-digit millisecond latency for NoSQL workloads, offering a serverless architecture that scales seamlessly to handle massive traffic spikes.

- **Amazon Redshift:** Acts as a powerful Data Warehouse (OLAP) that uses columnar storage to execute complex analytical queries on petabytes of data much faster than traditional databases.
- **RDS Deployment Lab:** To successfully launch an instance, you must select the Free Tier template to control costs, define a master user, and configure Security Group inbound rules to allow traffic through the database port.

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

